PADHAKU
Ganita Manjari Part II · Class 9 Mathematics

CHAPTER 11

The World of Algorithms

NCERT Exercise Solutions

Exercise Set 11.1	5 Q
Think and Reflect · 11.1 Adding Numbers	2 Q
Think and Reflect · 11.2.1 Computing the Divisors of a Number	1 Q
Exercise Set 11.2	5 Q
Exercise Set 11.3	2 Q
In-text question · 11.3.4 Āryabhaṭa's Division Algorithm	1 Q
End-of-Chapter Exercises	10 Q

✨ Try each question yourself first, then check the answer. 💡 answers are samples: write your own.

Independent study aid by Beginers Point. Not an official NCERT publication.

📝 Exercise Set 11.1

1 **Add two 4-digit numbers using the steps we have written down. Follow the steps precisely; do not perform any action that is not explicitly mentioned. Are you able to obtain the correct result?**

✔ ANSWER

Example 1: 4567 + 2345

- Step 2: 7 + 5 = 12 (more than 10) → write 2, carry = 1
- Step 3: 6 + 4 + 1 = 11 → write 1, carry = 1
- Step 3 again: 5 + 3 + 1 = 9 (less than 10) → write 9, carry = 0
- Step 3 again: 4 + 2 + 0 = 6 → write 6, carry = 0
- Step 5: carry is 0, nothing to add.

Result **6912**, which is correct.

Example 2: 4565 + 2345

- Step 2: 5 + 5 = 10. The steps only say what to do if the sum is **less than 10** or **more than 10**. Nothing is said for a sum **equal to 10**, so we get stuck!

So we get the correct result only when no column adds up to exactly 10. To fix it, Step 2 and Step 3 should say "10 or more" instead of "more than 10".

2 **What happens if you add a 5-digit number to a 3-digit number? Do our steps handle this**

situation correctly?

✔ ANSWER

Example: 12345 + 678.

After 3 columns, the 3-digit number has no digits left, but Step 3 says "add the two digits" and there is only one digit.

The steps do not say what to do, so they **do not handle it correctly** as written.

Fix: when one number runs out of digits, treat its missing digits as 0 (or just add the carry to the remaining digit).

With this fix: 12345 + 678 = **13023**.

3 **Why is it important to align the columns from right to left?**

✔ ANSWER

Aligning from the right puts units under units, tens under tens, hundreds under hundreds, and so on.

We may only add digits of the **same place value**.

If we aligned from the left, e.g. 473 + 95, we would add 4 hundreds to 9 tens, which gives a wrong answer.

4 **In Step 3, why cannot the value of carry be more than 1?**

✔ ANSWER

The largest digits are 9 and 9, and the carry coming in is at most 1.

So the largest possible column sum = 9 + 9 + 1 = 19.

19 is less than 20, so the tens digit of any column sum is at most 1.

Hence **the carry can never be more than 1**.

5 **What happens if we do not include the fifth step in the algorithm? Give examples where the algorithm will work correctly and where it will fail.**

✔ ANSWER

Without Step 5, a carry from the last (leftmost) column is lost.

Works: 1234 + 4321 → 5555 (no carry at the end) ✓.

Fails: 473 + 695 → the last column gives 4 + 6 + 1 = 11; we write 1 and carry 1, but the carry is never written, so we get 168 instead of **1168** ×.

So the algorithm works only when the last column has no carry.

🤔 Think and Reflect · 11.1 Adding Numbers

1 **See if you can complete the argument about grouping by units, tens, hundreds, ... to justify why the addition algorithm works.**

✔ ANSWER

Each number is a sum of groups: e.g. 473 = 4 hundreds + 7 tens + 3 units, 695 = 6 hundreds + 9 tens + 5 units.

Adding the numbers = adding units to units, tens to tens, hundreds to hundreds (we can add in any order and grouping).

- Units: 3 + 5 = 8 units.
- Tens: 7 + 9 = 16 tens = 1 hundred + 6 tens. The new hundred is moved (carried) to the hundreds group.
- Hundreds: 4 + 6 + 1 = 11 hundreds = 1 thousand + 1 hundred; the thousand is carried.

Total = 1 thousand + 1 hundred + 6 tens + 8 units = **1168**.

Since 10 of any group make exactly 1 of the next group, carrying keeps the total unchanged. So adding digit by digit with carries always gives the true sum.

2 **How would you modify the algorithm to add two decimal fractions?**

✔ ANSWER

- Write the numbers one below the other so that the **decimal points are in line** (not the rightmost digits).
- If one number has fewer digits after the point, fill the empty places with 0s.
- Add column by column from right to left with carries, exactly as before.
- Put the decimal point in the answer directly below the other decimal points.

Example: 3.75 + 12.6 = 3.75 + 12.60 = **16.35**.

🤔 Think and Reflect · 11.2.1 Computing the Divisors of a Number

? Try to execute the algorithm to compute the divisors of 15, 135, and 775. How does the amount of work increase as the numbers grow?

✔ ANSWER

Check every j from 1 to n and keep those that divide n:

- divisors(15) = **[1, 3, 5, 15]** (15 checks)
- divisors(135) = **[1, 3, 5, 9, 15, 27, 45, 135]** (135 checks)
- divisors(775) = **[1, 5, 25, 31, 155, 775]** (775 checks)

The work grows in proportion to the **value** of the number: for n we need n checks. So a number 10 times bigger needs about 10 times as much work.

Exercise Set 11.2

1 (i) **List 1 and List 2 are two lists of numbers in increasing order. Write an algorithm to find elements in List 1 that are not present in List 2.**

✔ ANSWER

Algorithm only-in-List-1:

1. Start with an empty list result.
2. For each number x in List 1:
   - if x does not appear in List 2, add x to result.
3. Report result.

Example: List 1 = [1, 3, 5, 15], List 2 = [1, 5, 25] → result **[3, 15]**.

(Faster, using the order: keep a pointer in each list; if the List 1 number is smaller, add it to result and move on in List 1; if equal, move on in both; if bigger, move on in List 2.)

1 (ii) **Write an algorithm to find elements in List 2 that are not present in List 1.**

✔ ANSWER

Same algorithm with the lists swapped:

1. Start with an empty list result.
2. For each number y in List 2:
   - if y does not appear in List 1, add y to result.
3. Report result.

Example: List 1 = [1, 3, 5, 15], List 2 = [1, 5, 25] → result **[25]**.

2 **Describe an algorithm to compute the least common multiple (lcm) of two numbers.**

✔ ANSWER

Algorithm to find lcm(m, n):

1. Let big = max(m, n) and small = min(m, n).
2. Let k = big.
3. If small divides k, report k as lcm(m, n).
4. Otherwise, set k = k + big and go back to Step 3.

(We only check multiples of the bigger number; the first one that the smaller number divides is the lcm.)

Example: lcm(6, 8): check 8, 16, 24 → 6 divides 24, so lcm = **24**.

Another way: lcm(m, n) = (m × n) ÷ gcd(m, n), using any gcd algorithm.

3 (i) **Divisors occur in pairs, e.g. divisors of 18 are (1, 18), (2, 9), (3, 6). If we write out divisors in pairs, how many numbers do we have to examine between 1 and n to find all the divisors of n?**

✔ ANSWER

In each pair (j, n ÷ j), the smaller number j is at most √n (because if both were bigger than √n, their product would be bigger than n).

So we only check j = 1, 2, 3, ... up to √n; each j that divides n gives the pair (j, n ÷ j).

We examine only about **√n numbers** instead of n.

Example: n = 18, √18 ≈ 4.2, so we check 1, 2, 3, 4 and get (1, 18), (2, 9), (3, 6).

3 (ii) **If we list out the divisors in pairs, will our gcd algorithm still work in the manner we have described?**

✔ ANSWER

Not as described. Listed in pairs, the divisors are not in increasing order, e.g. [1, 18, 2, 9, 3, 6].

Then the list of common divisors is also not in increasing order, so the rightmost element need not be the gcd.

Example: common divisors of 18 and 12 in this order: [1, 2, 6, 3] → rightmost is 3, but gcd = 6.

Fix: sort the list first (or report the largest element instead of the rightmost).

📝 Exercise Set 11.3

1 **How would our original algorithm change if we computed the divisors of n by examining**

the numbers from 1 to n in reverse order, from n down to 1?

✔ ANSWER

The lists of divisors would be in **decreasing** order, e.g. divisors(18) = [18, 9, 6, 3, 2, 1].

The list of common divisors would also be in decreasing order.

So in Step 5 we must report the **leftmost (first)** element of common-divisors as the gcd, instead of the rightmost.

Example: common divisors of 375 and 825 = [75, 25, 15, 5, 3, 1] → gcd = 75.

2 **What about the last algorithm described above? What happens when we look at common divisors starting from min(m, n) and work backwards to 1?**

✔ ANSWER

Going downwards, the **first** k that divides both m and n is the largest common divisor, i.e. the gcd.

So we can **stop at once** when we find it; we do not need most-recent-common-divisor or to scan the rest.

Example: m = 6, n = 12: start at k = 6; 6 divides both → gcd = **6** after just one check.

In the worst case (gcd = 1) we still check all numbers down to 1, but usually it is faster.

In-text question · 11.3.4 Āryabhaṭa's Division Algorithm

? **Check that only two reduction steps are needed for any problem of the form gcd(2k + 1, 2).**

✔ ANSWER

(2k + 1) mod 2 = 1, because 2k + 1 = 2 × k + 1.

- Reduction 1: gcd(2k + 1, 2) → gcd(2, 1)
- Reduction 2: 2 mod 1 = 0, so gcd(2, 1) → gcd(1, 0)
- n = 0, so the answer is 1.

So **always 2 reductions**, answer **1**, whatever k is (e.g. gcd(99, 2), gcd(1001, 2)).

End-of-Chapter Exercises

1 (i) **Compute gcd(375, 825) using the improved version of Euclid's algorithm.**

✔ ANSWER

375 < 825, so compute gcd(825, 375).

- 825 = 2 × 375 + 75 → gcd(375, 75)
- 375 = 5 × 75 + 0 → gcd(75, 0)

gcd(375, 825) = **75**

1 (ii) **gcd(51000, 81000)**

✔ ANSWER

51000 < 81000, so compute gcd(81000, 51000).

- 81000 = 1 × 51000 + 30000 → gcd(51000, 30000)
- 51000 = 1 × 30000 + 21000 → gcd(30000, 21000)
- 30000 = 1 × 21000 + 9000 → gcd(21000, 9000)
- 21000 = 2 × 9000 + 3000 → gcd(9000, 3000)
- 9000 = 3 × 3000 + 0 → gcd(3000, 0)

gcd(51000, 81000) = **3000**

1 (iii) gcd(1789287, 237656)

✔ **ANSWER**

- 1789287 = 7 × 237656 + 125695
- 237656 = 1 × 125695 + 111961
- 125695 = 1 × 111961 + 13734
- 111961 = 8 × 13734 + 2089
- 13734 = 6 × 2089 + 1200
- 2089 = 1 × 1200 + 889
- 1200 = 1 × 889 + 311
- 889 = 2 × 311 + 267
- 311 = 1 × 267 + 44
- 267 = 6 × 44 + 3
- 44 = 14 × 3 + 2
- 3 = 1 × 2 + 1
- 2 = 2 × 1 + 0 → gcd(1, 0)

gcd(1789287, 237656) = **1** (the numbers are co-prime)

1 (iv) **gcd(2587392, 157656)**

✔ ANSWER

- 2587392 = 16 × 157656 + 64896
- 157656 = 2 × 64896 + 27864
- 64896 = 2 × 27864 + 9168
- 27864 = 3 × 9168 + 360
- 9168 = 25 × 360 + 168
- 360 = 2 × 168 + 24
- 168 = 7 × 24 + 0 → gcd(24, 0)

gcd(2587392, 157656) = **24**

2 **Assume that m ≥ n. Verify that d divides m and n if and only if d divides both n and m mod n.**

✔ ANSWER

Let m = qn + r, where q is the quotient and r = m mod n.

1. Suppose d divides m and n: m = ad, n = bd. Then r = m − qn = ad − qbd = (a − qb)d. So d divides r = m mod n (and n).

2. Suppose d divides n and r: n = xd, r = yd. Then m = qn + r = qxd + yd = (qx + y)d. So d divides m (and n).

Hence **d divides m and n ⇔ d divides n and m mod n**. So both pairs have the same common divisors and gcd(m, n) = gcd(n, m mod n).

3 **Write an algorithm prime(n) to check if n is prime. (Hint: a prime p has exactly two distinct factors, 1 and p. Use divisors(n).)**

✔ ANSWER

Algorithm prime(n):

1. Let list-of-divisors be divisors(n).
2. If list-of-divisors has exactly 2 elements (they are 1 and n), report **"n is prime"**.
3. Otherwise, report **"n is not prime"**.

Examples: divisors(13) = [1, 13] → prime. divisors(15) = [1, 3, 5, 15] → not prime. divisors(1) = [1] → not prime (only one divisor).

4 **Write an algorithm primedivisors(n) to compute the list of divisors of n that are prime numbers. (Hint: compute divisors(n) and filter out the primes.)**

✔ ANSWER

Algorithm primedivisors(n):

1. Let list-of-divisors be divisors(n).
2. Start with an empty list prime-divisors.
3. For each number d in list-of-divisors:

- if prime(d) says d is prime, add d to prime-divisors.

4. Report prime-divisors.

Example: divisors(18) = [1, 2, 3, 6, 9, 18] → primedivisors(18) = **[2, 3]**.

5 **Try to write an algorithm to compute the prime factorisation of a number.**

✔ ANSWER

Algorithm to find the prime factorisation of n (n ≥ 2):

1. Start with an empty list factors. Let p = 2.
2. Repeat while n > 1:
   - Set count = 0.
   - While p divides n: set n = n ÷ p and count = count + 1.
   - If count > 0, add the pair (p, count) to factors.
   - Set p = p + 1.
3. Report factors.

Example: n = 180 → 180 ÷ 2 ÷ 2 = 45 (2 twice), 45 ÷ 3 ÷ 3 = 5 (3 twice), 5 ÷ 5 = 1 (5 once) → **[(2, 2), (3, 2), (5, 1)]**.

(Only primes divide in Step 2, because all smaller factors have already been divided out.)

5 (i) **The prime factorisation of 180 is $2^2 \times 3^2 \times 5^1$. How would you represent this?**

💡 SAMPLE ANSWER

As a list of (prime, power) pairs in increasing order of the primes:

[(2, 2), (3, 2), (5, 1)]

Each pair tells which prime appears and how many times.

5 (ii) How would you compare the prime factorisations of two numbers?

💡 **SAMPLE ANSWER**

Go through both lists of (prime, power) pairs (they are in increasing order of primes):

- For a prime that appears in **both** lists, take the **smaller** power and add it to the gcd list.
- A prime that is in only one list is not in the gcd.

gcd = product of these primes raised to these powers.

Example: 180 = [(2, 2), (3, 2), (5, 1)], 84 = $2^2 \times 3 \times 7$ = [(2, 2), (3, 1), (7, 1)].

Common: (2, 2) and (3, 1) → gcd = $2^2 \times 3$ = **12**.

(For the lcm, take every prime with the larger power.)

🎉 **Chapter 11 done! Revise the answers once more tomorrow.**